Analysis of the DBSCAN and K-DBSCAN Algorithms

Philip Loewen, Arusan Surendiran

This report provides fully reproducible code for the DBSCAN and K-DBSCAN analysis. It includes the manual implementations of the algorithms, how the simulated datasets were created, and the comparisons for execution time and clustering quality (DB and SI metrics). The document includes all figures and tables used in the final report.

Table of contents

1 DBSCAN Algorithm **2**

2 Simulation of Datasets **3**
2.1 Execution and Comparison 5
2.2 Visualization 5

3 K-means++ Algorithm **7**
3.1 Implementation 7
3.2 Simulation and Evaluation 9
3.3 Visualization 9

4 K-DBSCAN Algorithm **10**
4.1 Implementation 10
4.2 Application on Simulated Data 14

5 Comparison of DBSCAN and K-DBSCAN **15**
5.1 Optimized Algorithms for Computation 15
5.2 Clustering Quality Metrics 21
5.3 Simulation and Performance Evaluation 22
5.4 Visualizations 25

1 DBSCAN Algorithm

This section defines the manual implementation of the DBSCAN algorithm. It utilizes a queue in data structure and a pairwise distance matrix to handle cluster expansion efficiently in R.

```
manual_dbscan <- function(data, eps, minPts) {
  n <- nrow(data)
  cluster_assignment <- rep(0, n)  # 0 = unvisited, -1 = noise
  cluster_id <- 0

  # Compute pairwise Euclidean distances to define epsilon-neighborhoods
  dist_mat <- as.matrix(dist(data, method = "euclidean"))

  for (i in 1:n) {
    # Skip point if it has already been visited
    if (cluster_assignment[i] != 0) next

    # Retrieve the epsilon-neighborhood for point i
    neighbors <- which(dist_mat[i, ] <= eps)

    # Check core point condition
    if (length(neighbors) < minPts) {
      cluster_assignment[i] <- -1 # Temporarily classify as a noise point
    } else {
      # Core point verified: create a new density-based cluster
      cluster_id <- cluster_id + 1
      cluster_assignment[i] <- cluster_id

      # Identify directly density-reachable points to start the chain
      seeds <- setdiff(neighbors, i)

      # Initialize a queue to establish density-connected points
      queue <- integer(n)
      queue_len <- length(seeds)

      if (queue_len > 0) {
        queue[1:queue_len] <- seeds
      }

      head <- 1
      tail <- queue_len
```

```r
      # Expand the cluster to satisfy the Maximality requirement
      while (head <= tail) {
        current_p <- queue[head]
        head <- head + 1

        # If previously classified as noise, it is a border point
        if (cluster_assignment[current_p] == -1) {
          cluster_assignment[current_p] <- cluster_id
        }

        # If unvisited, process for density-reachability
        if (cluster_assignment[current_p] == 0) {
          cluster_assignment[current_p] <- cluster_id

          # Retrieve the epsilon-neighborhood of the current point
          new_neighbors <- which(dist_mat[current_p, ] <= eps)

          # If this point is also a core point, expand the reachable chain
          if (length(new_neighbors) >= minPts) {
            # Extract only unvisited or noise points
            valid_new <- new_neighbors[cluster_assignment[new_neighbors] <= 0]
            num_new <- length(valid_new)

            if (num_new > 0) {
              queue[(tail + 1):(tail + num_new)] <- valid_new
              tail <- tail + num_new
            }
          }
        }
      }
    }
  }
  return(cluster_assignment)
}
```

2 Simulation of Datasets

The simulation evaluates the algorithm's performance on non-linear geometries (sine curves, rings, and arcs) and its ability to distinguish clusters from background noise (Kuchciak, 2024). We demonstrate the advantages of DBSCAN clustering in comparison to k-means clustering.

```r
library(dbscan)
```

```
Attaching package: 'dbscan'

The following object is masked from 'package:stats':

    as.dendrogram
```

```r
library(ggplot2)
library(MASS)

set.seed(2201)

# Define the cluster centers and sizes
centers <- matrix(c(25, 25, -25, -25, 25, -20, -20, 20, -3, -7), ncol = 2,
                  byrow = TRUE)
size <- 1000

# Generate synthetic clusters with different characteristics
cluster1 <- mvrnorm(n = size, mu = centers[1,],
                    Sigma = matrix(c(9, 2, 2, 9), ncol = 2))
cluster2 <- mvrnorm(n = size, mu = centers[2,],
                    Sigma = matrix(c(12, 0.5, 0.5, 4), ncol = 2))
cluster3 <- mvrnorm(n = size, mu = centers[3,],
                    Sigma = matrix(c(6, 0.3, 0.3, 18), ncol = 2))
cluster4 <- mvrnorm(n = size, mu = centers[4,],
                    Sigma = matrix(c(15, 0.1, 0.1, 15), ncol = 2))
cluster5 <- mvrnorm(n = size, mu = centers[5,],
                    Sigma = matrix(c(7, 1.3, 1.3, 5), ncol = 2))

# Cluster 6: Sine curve distribution
x_values <- seq(-40, -20, length.out = 1000)
y_values <- sin(x_values) * 2 + rnorm(1000, mean = 0, sd = 0.2) # Adding some noise to the si
cluster6 <- cbind(x_values, y_values)

# Cluster 7: Points along an arc
theta <- runif(1000, min = pi, max = 2 * pi)
r <- 5
cluster7 <- cbind(r * cos(theta) + 5, r * sin(theta) + 15)
```

```r
# Cluster 8: Points in a "moon" shape
theta <- runif(1000, min = 0, max = pi)
r <- 10 + runif(1000, min = -1, max = 1)
cluster8 <- cbind(r * cos(theta) + 30, r * sin(theta))

# Cluster 9: Points in a "ring" shape around all other clusters
theta <- runif(1000, min = 0, max = 2 * pi)
r_inner <- 48
r_outer <- 52
r <- sqrt(runif(1000, min = r_inner^2, max = r_outer^2))
cluster9 <- cbind(r * cos(theta), r * sin(theta))

# Noise and Outliers
noise <- cbind(runif(250, -60, 60), runif(250, -60, 60))
outliers <- cbind(rnorm(100, 0, 20), rnorm(100, 0, 20))

data_full <- rbind(cluster1, cluster2, cluster3, cluster4, cluster5,
                   cluster6, cluster7, cluster8, cluster9, noise, outliers)
df_full <- as.data.frame(data_full)
colnames(df_full) <- c("X", "Y")
```

2.1 Execution and Comparison

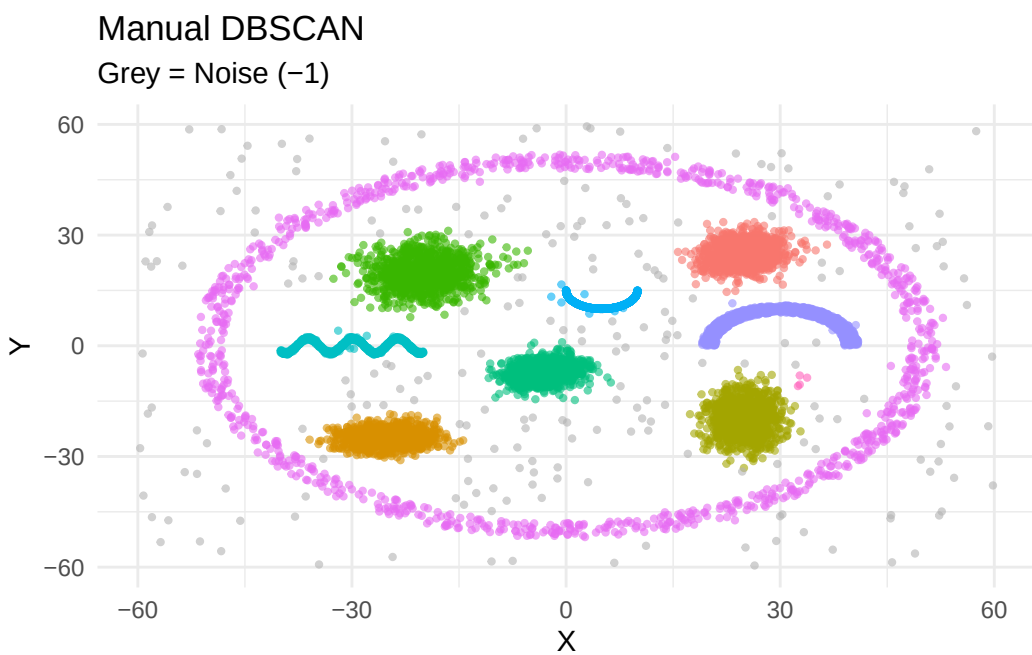
```r
set.seed(2201)

# Run Manual DBSCAN
results_dbscan <- manual_dbscan(df_full, eps = 2.5, minPts = 4)
df_full$Cluster <- as.factor(results_dbscan)
```

```r
# Run K-means for Comparison
km_result <- kmeans(df_full, centers = 9, nstart = 25)
df_full$Kmeans_Cluster <- as.factor(km_result$cluster)
```

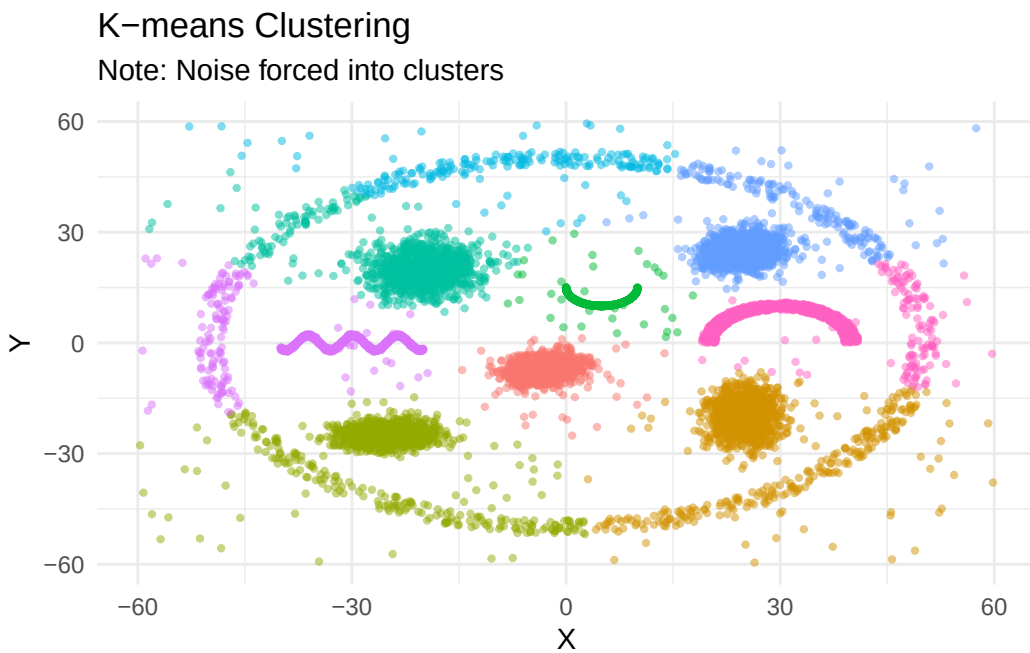
2.2 Visualization

```r
# Extract unique clusters excluding the noise (-1)
db_clusters <- unique(results_dbscan[results_dbscan != -1])
db_colors <- setNames(scales::hue_pal()(length(db_clusters)), db_clusters)
```

```r
db_palette <- c("-1" = "grey70", db_colors)


# DBSCAN Results
ggplot(df_full, aes(x = X, y = Y, color = Cluster)) +
  geom_point(alpha = 0.6, size = 0.8) +
  scale_color_manual(values = db_palette) +
  theme_minimal() +
  labs(title = "Manual DBSCAN", subtitle = "Grey = Noise (-1)") +
  theme(legend.position = "none")
```




```r
# K-means Results
ggplot(df_full, aes(x = X, y = Y, color = Kmeans_Cluster)) +
  geom_point(alpha = 0.5, size = 0.8) +
  theme_minimal() +
  labs(title = "K-means Clustering",
       subtitle = "Note: Noise forced into clusters") +
  theme(legend.position = "none")
```



The comparison above highlights the advantages of DBSCAN. It identified clusters of arbitrary shape and isolated noise points. In contrast, standard K-means forces noise into the nearest cluster, distorting the centroids and the overall structure of the data. K-means also notably did not pick up the shape of the elliptical ring around the other clusters.

3 K-means++ Algorithm

K-means++ improves upon the standard k-means algorithm by providing a specific method for choosing initial cluster centers. Standard k-means often relies on random initialization, which can lead to poor local optima. The k-means++ approach spreads out the initial centers before starting the iterative assignment and update steps.

3.1 Implementation

The following implementation includes the probabilistic seeding process and the iterative refinement loop.

```
# Function to compute Euclidean distance between a point and a set of centers
get_min_dist_sq <- function(data, centers) {
  # Calculate squared distances from each point to the nearest existing center
  apply(data, 1, function(point) {
    min(colSums((t(centers) - point)^2))
```

```r
  })
}

manual_kmeans_pp <- function(data, k, max_iter = 100) {
  data_mat <- as.matrix(data)
  n <- nrow(data_mat)

  # Step 1: K-means++ Seeding

  # Choose the first center uniformly at random
  centers <- matrix(NA, nrow = k, ncol = ncol(data_mat))
  centers[1, ] <- data_mat[sample(1:n, 1), ]

  if (k > 1) {
    for (i in 2:k) {
      # Compute squared distances to the nearest center already chosen
      dist_sq <- get_min_dist_sq(data_mat, centers[1:(i-1), , drop = FALSE])

      # Select next center with probability proportional to D(x)^2
      # This ensures centers are spread out across the feature space
      probs <- dist_sq / sum(dist_sq)
      next_idx <- sample(1:n, 1, prob = probs)
      centers[i, ] <- data_mat[next_idx, ]
    }
  }

  # Step 2: Standard K-means Iteration

  assignments <- rep(0, n)
  prev_assignments <- rep(-1, n)
  iter <- 0

  while (!all(assignments == prev_assignments) && iter < max_iter) {
    prev_assignments <- assignments
    iter <- iter + 1

    # Assignment Step: Assign each point to the nearest center
    dist_to_centers <- as.matrix(dist(rbind(centers, data_mat)))
    # Extract distances between points and centers only
    dist_matrix <- dist_to_centers[(k + 1):(n + k), 1:k]
    assignments <- apply(dist_matrix, 1, which.min)
```

```
    # Update Step: Recompute centers as the mean of assigned points
    for (j in 1:k) {
      if (sum(assignments == j) > 0) {
        centers[j, ] <- colMeans(data_mat[assignments == j, , drop = FALSE])
      }
    }
  }

  return(list(cluster = assignments, centers = centers, iterations = iter))
}
```

3.2 Simulation and Evaluation

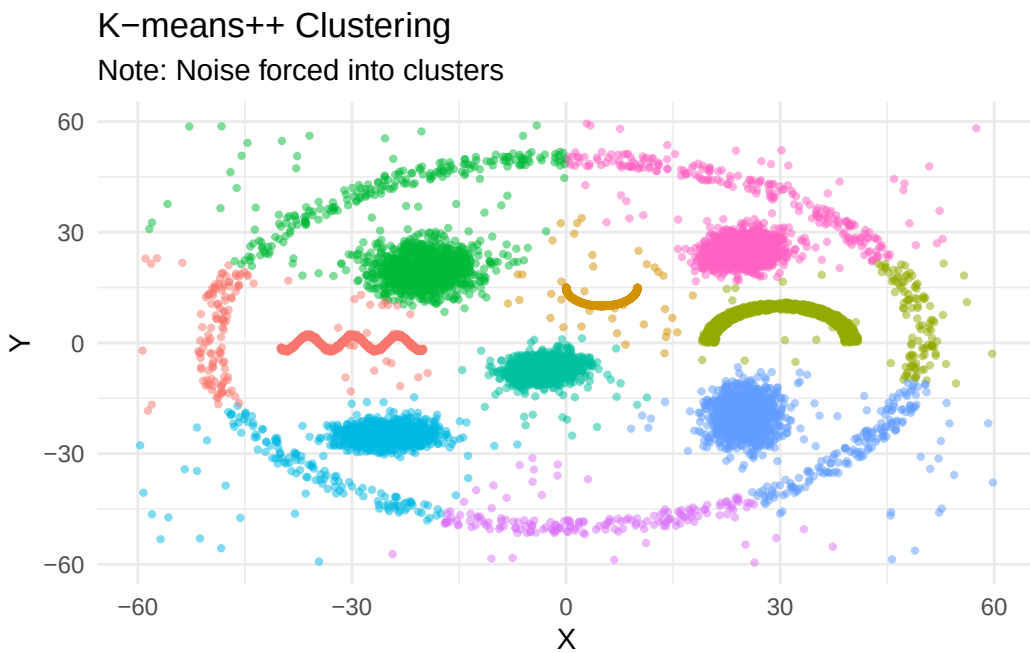
We reuse the previous synthetic dataset to evaluate how effectively k-means++ identifies the underlying structure.

```
set.seed(2201)
# Run Manual K-means++
# We choose k=9 to match the number of intended clusters in the generation
k_clusters <- 9
results_kmpp <- manual_kmeans_pp(df_full[, c("X", "Y")], k = k_clusters)
df_full$Kmpp_Cluster <- as.factor(results_kmpp$cluster)

centers_df <- as.data.frame(results_kmpp$centers)
colnames(centers_df) <- c("X", "Y")
```

3.3 Visualization

The plot below displays the final cluster assignments.

```
# Kmpp-means Results
ggplot(df_full, aes(x = X, y = Y, color = Kmpp_Cluster)) +
  geom_point(alpha = 0.5, size = 0.8) +
  theme_minimal() +
  labs(title = "K-means++ Clustering",
       subtitle = "Note: Noise forced into clusters") +
  theme(legend.position = "none")
```



4 K-DBSCAN Algorithm

The primary goal of the K-DBSCAN hybrid approach is to process data in localized segments and then unify density-connected clusters that cross the boundaries of those segments. By partitioning the space with K-means, the algorithm only performs exhaustive distance checks between points within the same group or between adjacent groups.

4.1 Implementation

The following implementation integrates the `manual_dbscan` and `manual_kmeans_pp` methods.

```
single_link_dist <- function(clu_1, clu_2) {
  clu_1 <- as.matrix(clu_1)
  clu_2 <- as.matrix(clu_2)

  if (nrow(clu_1) == 0 || nrow(clu_2) == 0) {
    return(Inf)
  }

  dist_mat <- as.matrix(dist(rbind(clu_1, clu_2)))
```

```r
  n1 <- nrow(clu_1)
  n2 <- nrow(clu_2)
  return(min(dist_mat[1:n1, (n1 + 1):(n1 + n2), drop = FALSE]))
}

cluster_dist <- function(clu_1, clu_2) {
  clu_1 <- as.matrix(clu_1)
  clu_2 <- as.matrix(clu_2)

  if (nrow(clu_1) == 0 || nrow(clu_2) == 0) {
    return(Inf)
  }

  center_1 <- colMeans(clu_1)
  center_2 <- colMeans(clu_2)

  dis_KCij <- sqrt(sum((center_1 - center_2)^2))
  dis_ri <- max(
    sqrt(rowSums((clu_1 - matrix(center_1, nrow = nrow(clu_1),
                                 ncol = ncol(clu_1), byrow = TRUE))^2)))
  dis_rj <- max(
    sqrt(rowSums((clu_2 - matrix(center_2, nrow = nrow(clu_2),
                                 ncol = ncol(clu_2), byrow = TRUE))^2)))

  return(dis_KCij - dis_ri - dis_rj)
}

k_dbscan <- function(D, k, max_iter, eps, minPts) {
  # Step 1: Normalization of D
  D_mat <- as.matrix(scale(D))
  n <- nrow(D_mat)

  if (n == 0) {
    return(integer(0))
  }

  # Step 2: Data grouping with K-means Algorithm
  km_results <- manual_kmeans_pp(D_mat, k = k, max_iter = max_iter)
  group_id <- km_results$cluster

  # Step 3: Run DBSCAN inside each K-means group
  group_points <- vector("list", k)
```

```r
numclu <- integer(k)

for (i in seq_len(k)) {
  group_idx <- which(group_id == i)
  if (length(group_idx) == 0) {
    group_points[[i]] <- list()
    next
  }

  group_data <- D_mat[group_idx, , drop = FALSE]
  local_labels <- manual_dbscan(group_data, eps, minPts)
  local_cluster_ids <- sort(unique(local_labels[local_labels > 0]))

  numclu[i] <- length(local_cluster_ids)
  group_points[[i]] <- lapply(local_cluster_ids, function(local_cluster) {
    group_idx[local_labels == local_cluster]
  })
}

# Step 4: Merge border clusters
flag <- vector("list", k)
for (i in seq_len(k)) {
  flag[[i]] <- rep(0L, numclu[i])
}

counter <- 1L

for (a in seq_len(k - 1L)) {
  for (b in seq_len(k)) {
    if (numclu[a] == 0L || numclu[b] == 0L) next

    dis_k <- cluster_dist(D_mat[group_id == a, , drop = FALSE],
                          D_mat[group_id == b, , drop = FALSE])
    if (dis_k > eps) next

    for (c in seq_len(numclu[a])) {
      for (d in seq_len(numclu[b])) {
        dis_db <- single_link_dist(
          D_mat[group_points[[a]][[c]], , drop = FALSE],
          D_mat[group_points[[b]][[d]], , drop = FALSE]
        )
```

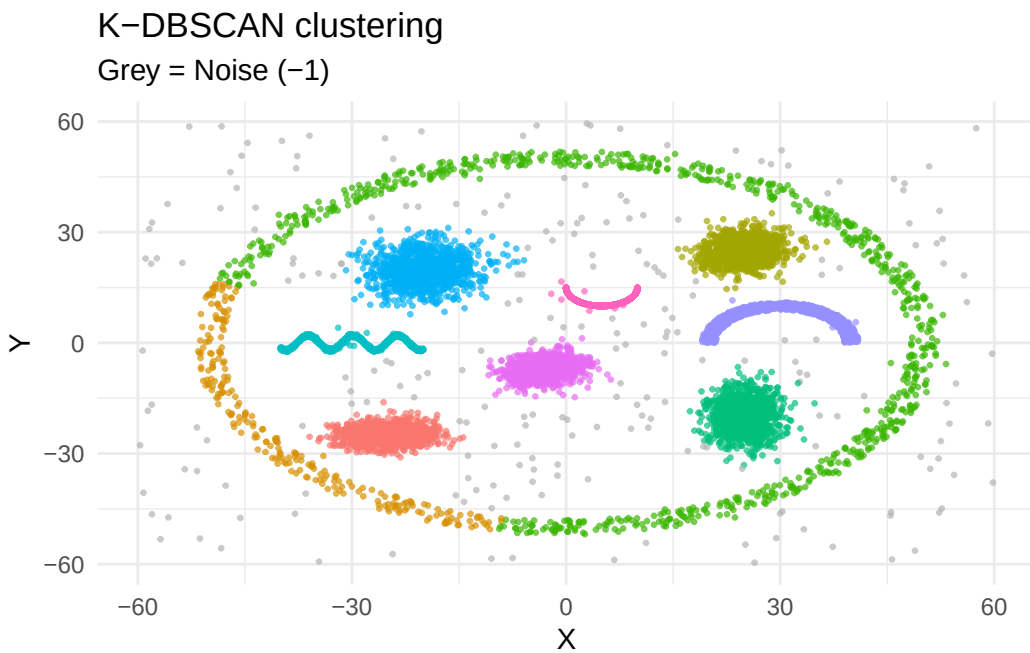
```
        if (dis_db <= eps) {
          if (flag[[a]][c] == 0L && flag[[b]][d] == 0L) {
            flag[[a]][c] <- counter
            flag[[b]][d] <- counter
            counter <- counter + 1L
          } else {
            new_flag <- max(flag[[a]][c], flag[[b]][d])
            flag[[a]][c] <- new_flag
            flag[[b]][d] <- new_flag
          }
        }
      }
    }
  }
}

for (i in seq_len(k)) {
  if (numclu[i] == 0L) next
  for (c in seq_len(numclu[i])) {
    if (flag[[i]][c] == 0L) {
      flag[[i]][c] <- counter
      counter <- counter + 1L
    }
  }
}

# Step 5: Run DBSCAN again inside each flagged component
final_labels <- rep(-1L, n)
final_cluster_id <- 0L

for (i in seq_len(counter - 1L)) {
  component_points <- integer(0)

  for (g in seq_len(k)) {
    if (numclu[g] == 0L) next
    for (c in seq_len(numclu[g])) {
      if (flag[[g]][c] == i) {
        component_points <- c(component_points, group_points[[g]][[c]])
      }
    }
  }
```

```r
    component_points <- sort(unique(component_points))
    if (length(component_points) == 0) next

    flagged_data <- D_mat[component_points, , drop = FALSE]
    flagged_result <- manual_dbscan(flagged_data, eps, minPts)

    for (local_cluster in sort(unique(flagged_result[flagged_result > 0]))) {
      final_cluster_id <- final_cluster_id + 1L
      final_labels[component_points[flagged_result == local_cluster]] <- final_cluster_id
    }
  }

  return(final_labels)
}
```

4.2 Application on Simulated Data

This section applies the K-DBSCAN algorithm to the synthetic data from earlier.

```r
results_kdbscan <- k_dbscan(df_full[, c("X", "Y")], 10, 4, 0.13, 6)
df_full$kdbs_cluster <- factor(results_kdbscan)

real_clusters <- sort(unique(results_kdbscan[results_kdbscan > 0]))

ggplot(df_full, aes(x = X, y = Y, color = kdbs_cluster)) +
  geom_point(size = 0.5, alpha = 0.7) +
  scale_color_manual(
    values = c("-1" = "grey70",
               setNames(scales::hue_pal()(length(real_clusters)),
                        as.character(real_clusters)))
  ) +
  labs(title = "K-DBSCAN clustering", subtitle = "Grey = Noise (-1)") +
  theme_minimal() +
  theme(legend.position = "none")
```



5 Comparison of DBSCAN and K-DBSCAN

The authors noted the challenge of running these algorithms on large datasets, citing computational capacity as a limit. For our cases, to show if there's any improvement in DBSCAN compared to the K-DBSCAN algorithm, we rewrite our above implementations with some changes. To enable our computers to run the algorithms on larger datasets, we optimize some of the key calculations that were originally so intensive, that it was beyond our computer's computing power. In specific, the distance matrix calculations for each point gets unmanageable once we reach a dataset of higher than 10,000 points.

Below are some re-wired versions of the coded algorithms. We use this to enable our computer to run the techniques, while still keeping the formulation of the algorithm the same.

5.1 Optimized Algorithms for Computation

```
find_uf <- function(parent, x) {
  while (parent[x] != x) x <- parent[x]
  return(x)
}
# Imporved Union Find methodology
union_uf <- function(parent, x, y) {
```

```r
  rx <- find_uf(parent, x)
  ry <- find_uf(parent, y)
  if (rx != ry) parent[rx] <- ry
  return(parent)
}

# find the max reach (radius) of a cluster
get_radius <- function(data_mat, center_vec) {
  if (nrow(data_mat) == 0) return(0)
  diffs <- sweep(data_mat, 2, center_vec, "-")
  return(max(sqrt(rowSums(diffs^2))))
}

# completely replaces single_link_dist and as.matrix(dist()).
# uses a linear algebra trick: (A-B)^2 = A^2 + B^2 - 2AB to compute distances
fast_min_dist <- function(A, B) {
  A <- as.matrix(A)
  B <- as.matrix(B)
  if (nrow(A) == 0 || nrow(B) == 0) return(Inf)

  D2 <- outer(rowSums(A^2), rowSums(B^2), "+") - 2 * tcrossprod(A, B)
  D2[D2 < 0] <- 0
  return(sqrt(min(D2)))
}

manual_dbscan_opt <- function(D, eps, minPts) {
  D <- as.matrix(D)
  n <- nrow(D)
  if (n == 0) return(integer(0))

  labels <- rep(0L, n)
  cluster_id <- 0L

  # rather than calling dist() on everything upfront, we calculate distances
  # point-by-point
  for (i in seq_len(n)) {
    if (labels[i] != 0L) next

    diffs <- sweep(D, 2, D[i, ], "-")
    dists <- sqrt(rowSums(diffs^2))
    neighbors <- which(dists <= eps)
```

```r
    if (length(neighbors) < minPts) {
      labels[i] <- -1L
    } else {
      cluster_id <- cluster_id + 1L
      labels[i] <- cluster_id
      seeds <- setdiff(neighbors, i)

      while (length(seeds) > 0) {
        cp <- seeds[1]
        seeds <- seeds[-1]

        if (labels[cp] == -1L) labels[cp] <- cluster_id
        if (labels[cp] != 0L) next

        labels[cp] <- cluster_id
        diffs_seed <- sweep(D, 2, D[cp, ], "-")
        dists_seed <- sqrt(rowSums(diffs_seed^2))
        seed_neighbors <- which(dists_seed <= eps)

        if (length(seed_neighbors) >= minPts) {
          new_seeds <- setdiff(seed_neighbors, c(seeds, i, cp))
          if (length(new_seeds) > 0) seeds <- c(seeds, new_seeds)
        }
      }
    }
  }
  return(labels)
}

k_dbscan_opt <- function(D, eps, minPts) {
  D_mat <- as.matrix(scale(D))
  n <- nrow(D_mat)
  if (n < 2) return(rep(-1L, n))

  k <- max(1, min(floor(sqrt(n)), n - 1))

  # use packaged k-means
  km <- suppressWarnings(kmeans(D_mat, centers = k, iter.max = 4))
  group_id <- km$cluster
  centers <- km$centers

  group_points <- vector("list", k)
```

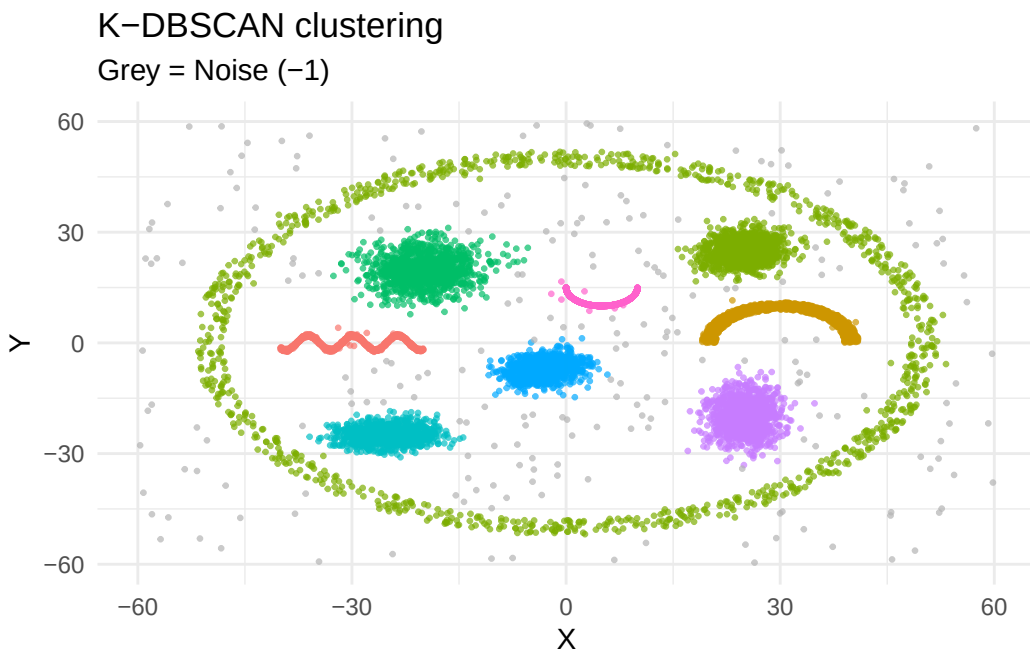
```r
numclu <- integer(k)
group_radii <- numeric(k)
sub_centers <- vector("list", k)
sub_radii <- vector("list", k)

for (i in seq_len(k)) {
  idx <- which(group_id == i)
  if (length(idx) == 0) next

  group_data <- D_mat[idx, , drop = FALSE]

  # calculating the boundary radius for the whole k-means partition
  group_radii[i] <- get_radius(group_data, centers[i, ])

  local_labels <- manual_dbscan_opt(group_data, eps, minPts)
  u_labels <- sort(unique(local_labels[local_labels > 0]))
  numclu[i] <- length(u_labels)

  group_points[[i]] <- lapply(u_labels, function(l) idx[local_labels == l])
  sub_centers[[i]] <- lapply(group_points[[i]],
                             function(p) colMeans(D_mat[p, , drop = FALSE]))

  # storing boundary radii for the internal sub-clusters
  sub_radii[[i]] <- mapply(function(p, ctr) get_radius(
    D_mat[p, , drop = FALSE], ctr
    ), group_points[[i]], sub_centers[[i]])
}

total_subclu <- sum(numclu)
if (total_subclu == 0) return(rep(-1L, n))

parent <- seq_len(total_subclu)

sub_id_map <- vector("list", k)
curr <- 0
for (g in seq_len(k)) {
  if(numclu[g] > 0) {
    sub_id_map[[g]] <- (curr + 1):(curr + numclu[g])
    curr <- curr + numclu[g]
  }
}
```

```r
# instead of checking every group against every other group,
# we only check groups whose maximum radii actually overlap.
g_dist <- as.matrix(dist(centers))
g_adj <- (g_dist - outer(group_radii, group_radii, "+")) <= eps
g_adj[lower.tri(g_adj, diag = TRUE)] <- FALSE
pairs <- which(g_adj, arr.ind = TRUE)

if (nrow(pairs) > 0) {
  for (idx in seq_len(nrow(pairs))) {
    g1 <- pairs[idx, 1]
    g2 <- pairs[idx, 2]

    for (c1 in seq_len(numclu[g1])) {
      for (c2 in seq_len(numclu[g2])) {
        d_dc <- sqrt(
          sum((sub_centers[[g1]][[c1]] - sub_centers[[g2]][[c2]])^2)
          )

        # even if the k-means groups overlap, we only run the  distance calculation
        # if the specific sub-clusters inside them overlap too.
        if ((d_dc - (sub_radii[[g1]][c1] + sub_radii[[g2]][c2])) <= eps) {

          # using the matrix computation instead of single_link_dist
          if (fast_min_dist(D_mat[group_points[[g1]][[c1]], ],
                            D_mat[group_points[[g2]][[c2]], ]) <= eps) {
            parent <- union_uf(
              parent, sub_id_map[[g1]][c1], sub_id_map[[g2]][c2]
              )
          }

        }
      }
    }
  }
}

# resolving final labels using the union-find structure instead of running a
# completely new pass of manual_dbscan on flagged components.
roots <- unique(vapply(seq_len(total_subclu), find_uf, integer(1),
                       parent = parent))
final_labels <- rep(-1L, n)
cid <- 0
```

```
  for (r in roots) {
    pts <- unlist(lapply(seq_len(k), function(g) {
      if(numclu[g] > 0) {
        matches <- which(vapply(sub_id_map[[g]],
                                find_uf, 1, parent=parent) == r)
        if (length(matches) > 0) return(unlist(group_points[[g]][matches]))
      }
      return(NULL)
    }))

    if (length(pts) > 0) {
      cid <- cid + 1
      final_labels[pts] <- cid
    }
  }
  return(final_labels)
}
```

The optimized 'k_dbscan' 's results do not vastly differ from the original implementation from above.

```
results_kdbscan <- k_dbscan_opt(df_full[, c("X", "Y")], 0.12, 5)
df_full$kdbs_cluster <- factor(results_kdbscan)

real_clusters <- sort(unique(results_kdbscan[results_kdbscan > 0]))

ggplot(df_full, aes(x = X, y = Y, color = kdbs_cluster)) +
  geom_point(size = 0.5, alpha = 0.7) +
  scale_color_manual(
    values = c("-1" = "grey70",
               setNames(scales::hue_pal()(length(real_clusters)),
                        as.character(real_clusters)))
  ) +
  labs(title = "K-DBSCAN clustering", subtitle = "Grey = Noise (-1)") +
  theme_minimal() +
  theme(legend.position = "none")
```



5.2 Clustering Quality Metrics

```
library(cluster)
library(clusterSim)

evaluate_packaged_metrics <- function(data, labels, max_sample = 5000) {
  # Filter out noise points (-1) and unassigned points (0)
  valid_idx <- labels > 0

  if (sum(valid_idx) == 0 || length(unique(labels[valid_idx])) < 2) {
    return(c(DB = NA, SI = NA))
  }

  data_c <- data[valid_idx, , drop = FALSE]
  labels_c <- labels[valid_idx]

  # Davies-Bouldin Index (FIXED: parameter is 'centrotypes')
  db_res <- index.DB(data_c, as.integer(labels_c), centrotypes = "centroids")
  db_val <- db_res$DB

  # Silhouette Index (Sampled for memory safety)
  if (nrow(data_c) > max_sample) {
```

```
    set.seed(42)
    sample_idx <- sample(seq_len(nrow(data_c)), max_sample)
    data_c <- data_c[sample_idx, ]
    labels_c <- labels_c[sample_idx]
  }

  dist_mat <- dist(data_c)
  sil_res <- silhouette(as.integer(labels_c), dist_mat)
  si_val <- mean(sil_res[, "sil_width"])

  return(c(DB = db_val, SI = si_val))
}
```

5.3 Simulation and Performance Evaluation

The following creates a simulation by generating the progressively larger datasets and executing both algorithms under identical parameters. It then aggregates the elapsed execution times alongside the Davies-Bouldin and Silhouette quality metrics.

```
# Simulation Parameters
test_sizes <- c(4000, 8000, 16000, 32000, 64000)
noise_ratio <- 0.10
eps_sim <- 0.15
minPts_sim <- 10

# Helper for constant noise density
get_noise_limit <- function(n) { 4 * sqrt(n / 4000) }


results_master <- data.frame()

for (n in test_sizes) {
  cat("Processing N =", n, "---")

  # Generate Data
  set.seed(n)
  limit <- get_noise_limit(n)
  n_noise <- floor(n * noise_ratio)
  n_signal <- n - n_noise
  pts_per_blob <- floor(n_signal / 4)
```

```r
  blob_centers <- matrix(c(-2,-2, 2,2, -2,2, 2,-2), ncol = 2, byrow = TRUE)
  signal_data <- do.call(rbind, lapply(1:4, function(i) {
    matrix(rnorm(pts_per_blob * 2,
                 mean = blob_centers[i,], sd = 0.4), ncol = 2)
  }))
  noise_data <- matrix(runif(n_noise * 2,
                             min = -limit, max = limit), ncol = 2)

  sim_data <- scale(rbind(signal_data, noise_data))
  sim_data <- sim_data[sample(nrow(sim_data)), ] # Shuffle

  # Benchmark & Run Standard DBSCAN
  cat("\n  Running Standard DBSCAN... ")
  t_db <- system.time({
    res_db <- manual_dbscan_opt(sim_data, eps_sim, minPts_sim)
  })
  m_db <- evaluate_packaged_metrics(sim_data, res_db)

  # Benchmark & Run K-DBSCAN
  cat("Running K-DBSCAN... ")
  t_kdb <- system.time({
    res_kdb <- k_dbscan_opt(sim_data, eps_sim, minPts_sim)
  })
  m_kdb <- evaluate_packaged_metrics(sim_data, res_kdb)

  row_data <- data.frame(
    N = n,
    DBSCAN_Time = t_db["elapsed"],
    K_DBSCAN_Time = t_kdb["elapsed"],
    DBSCAN_DB = m_db["DB"],
    K_DBSCAN_DB = m_kdb["DB"],
    DBSCAN_SI = m_db["SI"],
    K_DBSCAN_SI = m_kdb["SI"]
  )
  results_master <- rbind(results_master, row_data)
}
```

```
Processing N = 4000 ---
  Running Standard DBSCAN... Running K-DBSCAN... Processing N = 8000 ---
  Running Standard DBSCAN... Running K-DBSCAN... Processing N = 16000 ---
  Running Standard DBSCAN... Running K-DBSCAN... Processing N = 32000 ---
  Running Standard DBSCAN... Running K-DBSCAN... Processing N = 64000 ---
```

```
  Running Standard DBSCAN... Running K-DBSCAN...
```

```r
cat("EXECUTION PERFORMANCE & CLUSTER QUALITY REPORT\n")
```

```
EXECUTION PERFORMANCE & CLUSTER QUALITY REPORT
```

```r
print(results_master, row.names = FALSE)
```

```
     N DBSCAN_Time K_DBSCAN_Time DBSCAN_DB K_DBSCAN_DB DBSCAN_SI K_DBSCAN_SI
  4000       0.775         0.277 0.2025184   0.2015455 0.8730855   0.8735681
  8000       2.740         0.654 0.2048695   0.2028794 0.8717193   0.8732632
 16000      11.613         1.591 0.2025086   0.2016875 0.8732756   0.8737005
 32000      60.946         4.426 0.2919258   0.2167475 0.7272872   0.8075189
 64000     356.262        13.324 0.5985437   0.5713018 0.7318217   0.7618845
```

```r
library(knitr)
library(kableExtra)

# Create a data frame for the parameters
params_df <- data.frame(
  Parameter = c("Test Sizes (N)", "Epsilon (eps)", "Min. Points (minPts)"),
  Value = c("4000, 8000, 16000, 32000, 64000", "0.15", "10"),
  Description = c("Number of data points in each simulation",
                  "Reachability distance for DBSCAN",
                  "Minimum cluster density threshold")
)

params_df |>
  kable(booktabs = TRUE,
        caption = "Simulation Parameters for Performance Benchmarking") |>
  kable_styling(
    bootstrap_options = "striped",
    latex_options = c("striped", "hold_position"),
    full_width = FALSE
  ) |>
  column_spec(1, bold = TRUE)
```

Table 1: Simulation Parameters for Performance Benchmarking

Parameter	Value	Description
Test Sizes (N)	4000, 8000, 16000, 32000, 64000	Number of data points in each simulation
Epsilon (eps)	0.15	Reachability distance for DBSCAN
Min. Points (minPts)	10	Minimum cluster density threshold

5.4 Visualizations

```
df_lines <- data.frame(
  N = rep(results_master$N, 2),
  Time = c(results_master$DBSCAN_Time, results_master$K_DBSCAN_Time),
  Algorithm = factor(
    rep(c("Standard DBSCAN", "K-DBSCAN"), each = nrow(results_master)),
    levels = c("Standard DBSCAN", "K-DBSCAN")
  )
)

ggplot(df_lines, aes(x = N, y = Time, color = Algorithm, shape = Algorithm)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  scale_color_manual(values = c("Standard DBSCAN" = "firebrick",
                                "K-DBSCAN" = "dodgerblue4")) +
  scale_shape_manual(values = c("Standard DBSCAN" = 16, "K-DBSCAN" = 17)) +
  labs(
    title = "Comparing Standard DBSCAN vs K-DBSCAN for Large Datasets",
    x = "Number of Data Points (N)",
    y = "Execution Time (Seconds)"
  ) +
  scale_x_continuous(limits = c(min(df_lines$N), max(df_lines$N))) +
  scale_y_continuous(limits = c(0, max(df_lines$Time) * 1.1)) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = c(0.15, 0.85), # Positions legend in the top-left
    legend.title = element_blank(),
    legend.background = element_rect(fill = "white", color = NA),
    panel.grid.major = element_line(color = "lightgray", linetype = "dotted")
  )
```

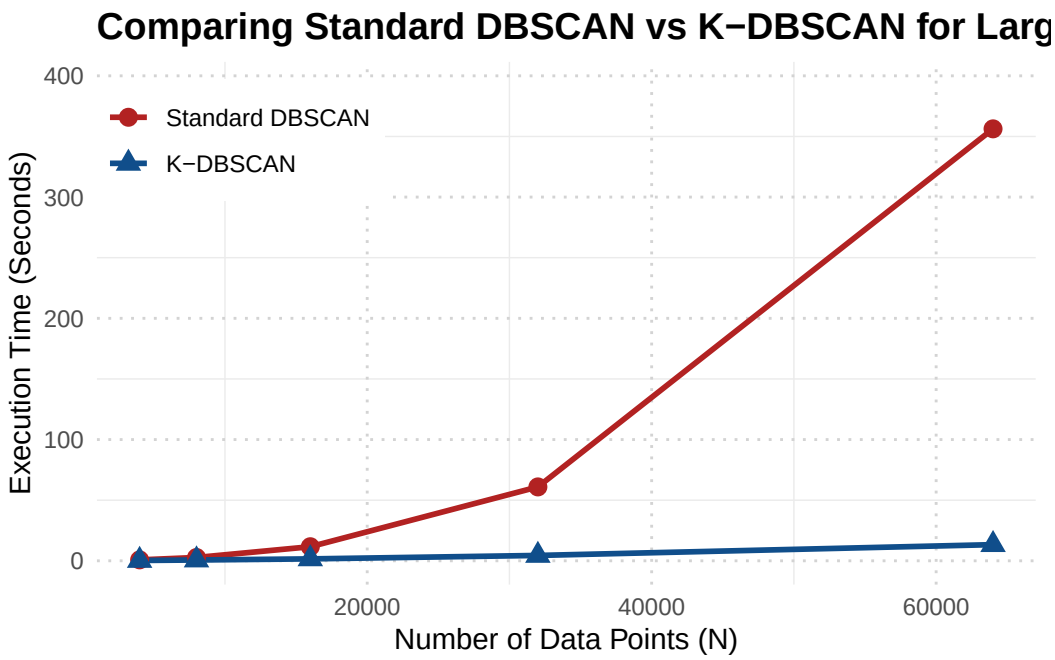
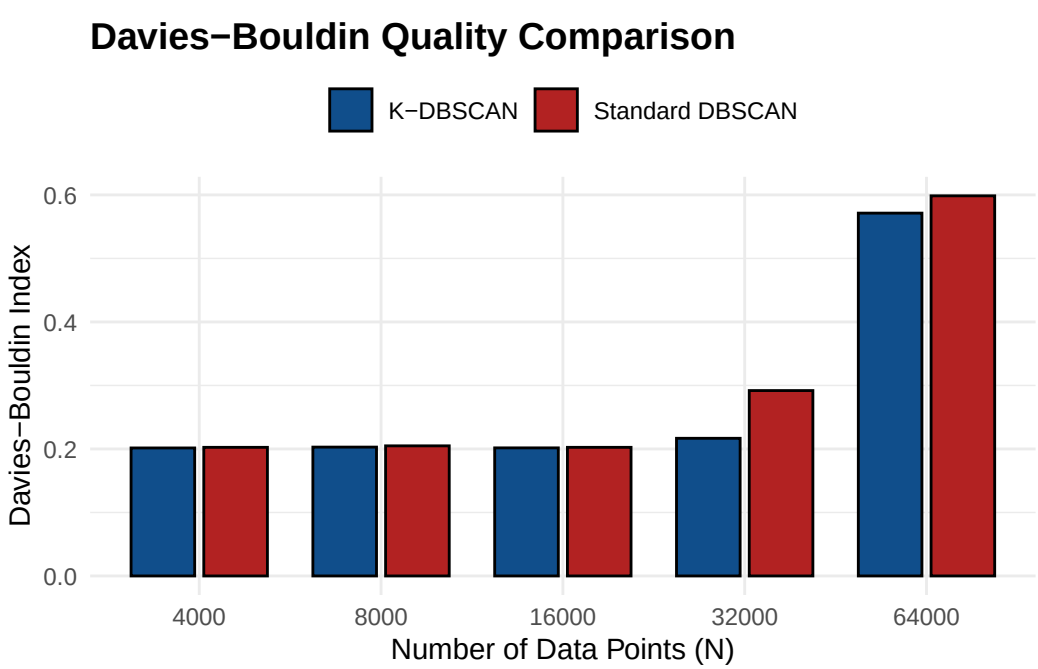


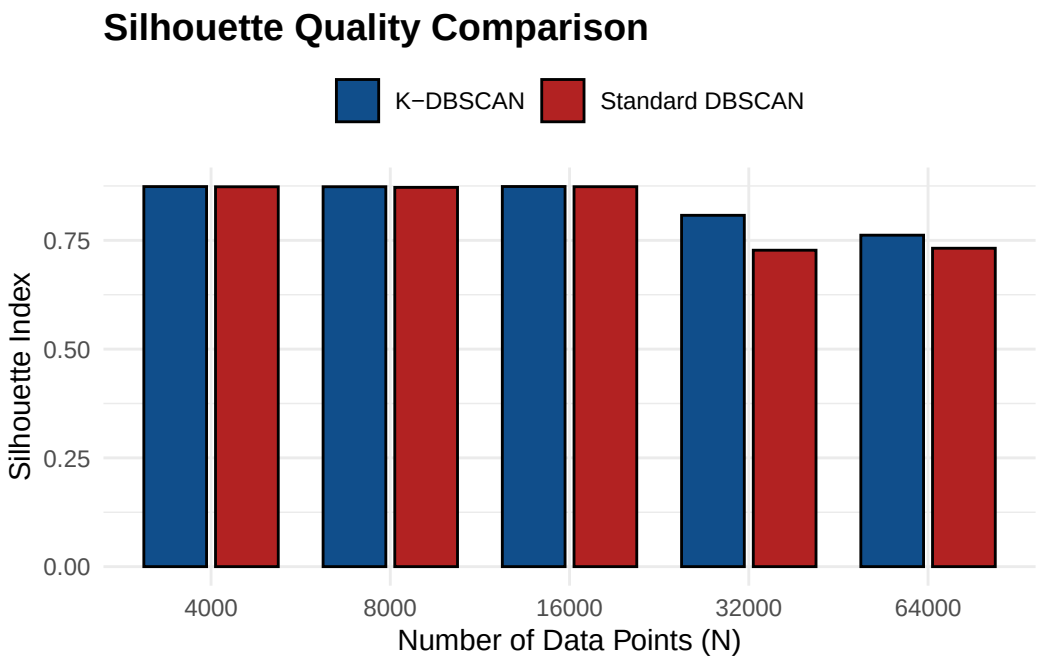
Figure 1: Diagram for comparison of execution time in the proposed method, K-DBSCAN, and standard DBSCAN.

```
# Davies-Bouldin Plot
df_db <- data.frame(
  N = rep(results_master$N, 2),
  Algorithm = rep(c("Standard DBSCAN", "K-DBSCAN"),
                  each = nrow(results_master)),
  Score = c(results_master$DBSCAN_DB, results_master$K_DBSCAN_DB)
)

plot_db <- ggplot(df_db, aes(x = factor(N), y = Score, fill = Algorithm)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.8),
           width = 0.7, color="black") +
  scale_fill_manual(values = c("Standard DBSCAN" = "firebrick",
                               "K-DBSCAN" = "dodgerblue4")) +
  labs(title = "Davies-Bouldin Quality Comparison",
       x = "Number of Data Points (N)",
       y = "Davies-Bouldin Index") +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", size = 14),
        legend.position = "top",
        legend.title = element_blank())
```

```r
# Silhouette Plot
df_si <- data.frame(
  N = rep(results_master$N, 2),
  Algorithm = rep(c("Standard DBSCAN", "K-DBSCAN"),
                  each = nrow(results_master)),
  Score = c(results_master$DBSCAN_SI, results_master$K_DBSCAN_SI)
)

# Plot Silhouette
plot_si <- ggplot(df_si, aes(x = factor(N), y = Score, fill = Algorithm)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.8),
           width = 0.7, color="black") +
  scale_fill_manual(values = c("Standard DBSCAN" = "firebrick",
                               "K-DBSCAN" = "dodgerblue4")) +
  labs(title = "Silhouette Quality Comparison",
       x = "Number of Data Points (N)",
       y = "Silhouette Index") +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", size = 14),
        legend.position = "top",
        legend.title = element_blank())

plot_db
```



```
plot_si
```


Silhouette Quality Comparison
K−DBSCAN
Standard DBSCAN
Silhouette Index
0.75
0.50
0.25
0.00
4000
8000
16000
32000
64000
Number of Data Points (N)